

Data Collection

Collecting data for training the ML model is the basic step in the machine learning pipeline. The predictions made by ML systems can only be as good as the data on which they have been trained.

1. **Inaccurate data-** The collected data could be unrelated to the problem statement.
2. **Missing data-** Sub-data could be missing. That could take the form of empty values in columns or missing images for some class of prediction.
3. **Data imbalance-** Some classes or categories in the data may have a variably high or low number of corresponding samples. As a result, they risk being under-represented in the model.
4. **Data bias-** Depending on how the data, subjects, and labels themselves are chosen, the model could propagate inherent biases on gender, politics, age, or region, for example. Data bias is difficult to detect and remove.

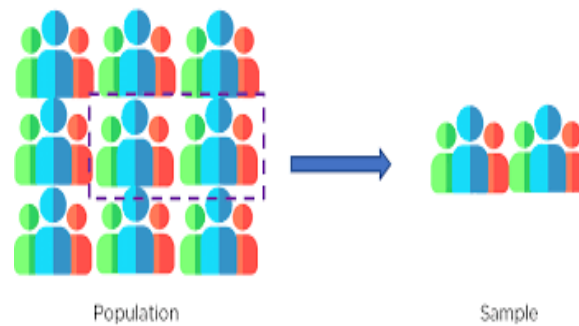
Population and sampling

population

1. In statistics, population is the entire set of items from which we can draw data for a statistical study.
2. It can be a group of individuals, a set of items, etc. It makes up the data pool for a study.
3. Population refers to data on your study of interest. It can be a group of individuals, objects, events, organizations, etc.
4. An example of a population would be the entire student body at a school. It would contain all the students who study in that school at the time of data collection. Depending on the problem statement, data from each of these students is collected. An example is the students who speak hindi among the students of a school.

Sample:

1. Sample is defined **as a smaller and more manageable representation of a larger group.**
2. A subset of a larger population that contains characteristics of that population.
3. A sample is used in statistical testing when the population size is too large for all members or observations to be included in the test.
4. The process of collecting data from a small subsection of the population and then using it to generalize over the entire set is called **sampling.**



Samples are used when:

- The population is too large to collect data.
- The data collected is not reliable.
- The population is hypothetical and is unlimited in size.

Take the example of a study that documents the results of a new medical procedure. It is unknown how the procedure will affect people across the globe, so a test group is used to find out how people react to it.

Data Types In ML

Type 1 cross sectional and time series data

The key difference between time series and cross-sectional data is that

Time-series	Cross-sectional
1. The time series data focuses on the same <u>variable</u> over a period	1. While the cross-sectional data focuses on <u>several variables</u> at the same point of time.
2. The time series data consist of observations of a single subject at multiple time intervals	2. The cross-sectional data consist of observations of many subjects at the same point in time.
3. Example: suppose time series data is the daily opening prices of a share over a half-yearly period.	3. Example: The value of opening prices of 10 selected stocks on a given date is an example of cross-sectional data.
4. Time series data focuses on observations of a single individual at different times usually at uniform intervals.	4. In cross sectional data, there are several variables at the same point in time. Data set with maximum temperature, humidity, wind speed of few cities on a single day is an example of a cross sectional data.
5. It is the data of the same variable over a period such as months, quarters, years etc.	

6. The time series data takes the form of x_t . The t represents the time. Below is an example of the profit of an organization over a period of 5 years' time. Profit is the variable that changes each year.	5. Another example is the sales revenue, sales volume, number of customers and expenses of an organization in the past month. Cross sectional data takes the form of x_i . Expanding the data from several months will convert the cross-sectional data to time series data.
--	--

Time-series

Year	Profit
2001	50000
2002	60000
2003	70000
2004	60000
2005	75000

Cross-sectional

City	Maximum Temperature	Humidity	Wind Speed
City A	29	60%	20mph
City B	27	65%	26mph
City C	30	60%	21mph

Type 2 univariate and multivariate data

1.Univariate

- Uni means **one and variate means variable**, so in univariate analysis, there is only one dependable variable.
- The objective of univariate analysis is to derive the data, define and summarize it, and analyze the pattern present in it.
- In a dataset, it explores each variable separately. It is possible for two kinds of variables- categorical and numerical.
- The description of patterns found in this type of data can be made by drawing conclusions using central tendency measures (mean, median and mode), dispersion or spread of data (range, minimum, maximum, quartiles, variance and standard deviation) and by using frequency distribution tables, histograms, pie charts, frequency polygon and bar charts.

2.Bi-variate

- This type of **data involves two different variables**. The analysis of this type of data deals with causes and relationships and the analysis is done to find out the relationship among the two variables.
- Bivariate data analysis involves comparisons, relationships, causes and explanations.
- These variables are often plotted on X and Y axis on the graph for better understanding of data and one of these variables is independent while the other is dependent.
- Bivariate data is when you are studying two variables.
- Example of bivariate data can be temperature and ice cream sales in summer season.

TEMPERATURE(IN CELSIUS)	ICE CREAM SALES
20	2000
25	2500
35	5000
43	7800

3.Multivariate data

- When **the data involves three or more variables**, it is categorized under multivariate.
- It is similar to bivariate but contains more than one dependent variable.
- Multivariate refers to multiple dependent variables that result in one outcome
- This means that a majority of our real-world problems are multivariate.
- The ways to perform analysis on this data depends on the goals to be achieved. Some of the techniques are regression analysis, path analysis, factor analysis and multivariate analysis of variance (MANOVA).
- Example of this type of data is suppose an advertiser wants to compare the popularity of four advertisements on a website, then their click rates could be measured for both men and women and relationships between variables can then be examined.

Variables Types in Machine Learning

1.Categorical:

1. categorical data represents characteristics.
2. Categorical data is the data that generally takes a limited number of possible values.
3. The data in the category need not be numerical, it can be textual in nature.
4. Categorical variables are usually represented as 'strings' or 'categories' and are finite in number.
5. All machine learning models are some kind of mathematical model that need numbers to work with. This is one of the primary reasons we need to pre-process the categorical data before we can feed it to machine learning models.

Examples:

The city where a person lives: delhi, mumbai, ahmedabad, bangalore, etc.

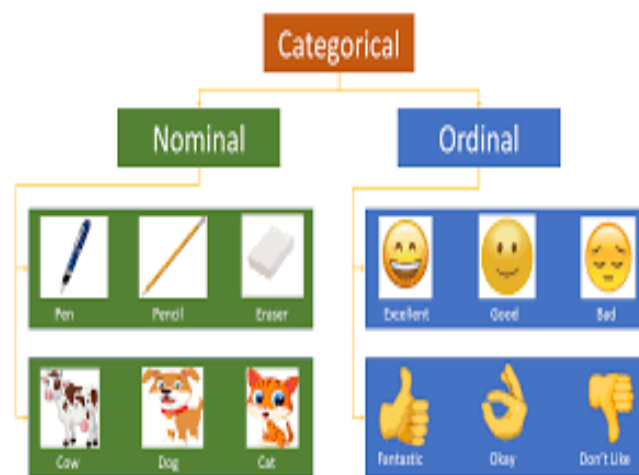
The department a person works in: finance, human resources, it, production.

The highest degree a person has: high school, diploma, bachelors, masters, phd.

The grades of a student: a+, a, b+, b, b- etc

2.Ordinal-

1. ordinal data is a categorical, statistical data type where the variables have natural, ordered categories and the distances between the categories are not known.
2. Ordinal data are discrete integers that can be ranked or sorted
3. A defining characteristic is that the distance between any two numbers is not known.



3.Ratio

1. Ratio- Ratio data is a form of quantitative (numeric) data.
2. It measures variables on a continuous scale, with an equal distance between adjacent values.
3. A distinguishing property of ratio data is that it has a 'true zero'.
4. *Income, height, weight, annual sales, market share, product defect rates, time to repurchase, unemployment rate, and crime rate are examples of ratio data*

4.Interval

1. Interval data is a type of quantitative (numerical) data.
2. It groups variables into categories and always uses some kind of ordered scale.
3. Furthermore, interval values are always ordered and separated using an equal measure of distance.
4. Temperature (in celsius or fahrenheit), mark grading, IQ test and CGPA.

Probability Distribution

Probability-An event is a set of outcomes (one or more) from an experiment. For example, when we toss a coin “Getting a Tail when tossing a coin is an event”, probability is the measure of occurrence of an event. It is defined as *Probability is the likelihood of an event occurring*. Probability is the mathematical study of uncertainty and also Many events cannot be predicted with total certainty.

1. Joint probability

1. Joint probability is the probability of two events occurring simultaneously.
2. The probability of event A and event B occurring together.
3. It is the probability of the intersection of two or more events written as $p(A \cap B)$.

2. conditional probability

1. The conditional probability of an event B is the probability that the event will occur given the knowledge that an event A has already occurred. It is denoted by $P(B|A)$.
2. Conditional probability is known as the possibility of an event or outcome happening, based on the existence of a previous event or outcome.

Broadly speaking, joint probability is the probability of two things* happening together: e.g., the probability that I wash my car, and it rains. Conditional probability is the probability of one thing happening, given that the other thing happens: e.g., the probability that, given that I wash my car, it rains.

Joint Probability Formula

$$\text{Joint Probability (JP)} = P(A) \times P(B)$$



Conditional probability: Bayes' Theorem

$$P(A|B) = \frac{P(B|A) P(A)}{P(B)}$$

© Byjus.com

Bayes Theorem

Bayes Theorem- Describes the probability of an event, based on prior knowledge of conditions that might be related to the event.

Conditional probability: Bayes' Theorem

$$P(A|B) = \frac{P(B|A) P(A)}{P(B)}$$

© Byjus.com

where a and b are events and $p(B) = 0$

$P(A|B)$ is a conditional probability: the probability of event A occurring given that B is true. It is also called the posterior probability of A given B.

$P(B|A)$ is also a conditional probability:

the probability of event B occurring given that A is true.

It can also be interpreted as the likelihood of A given a fixed B because

$P(B|A) = L(A|B)$.

$P(A)$ and $P(B)$ are the probabilities of observing A and B respectively without any given conditions; they are known as the prior probability and marginal probability.

- **Prior probability, in Bayesian statistics, is the probability of an event before new data is collected.**
- **The probability of one event in the presence of all (or a subset of) outcomes of the other random variable is called the marginal probability**

Probability distribution

- A probability distribution is a mathematical function that describes the probability of different possible values of a variable.
- In probability theory and statistics, a probability distribution is the mathematical function that gives the probabilities of occurrence of different possible outcomes for an experiment.
- For instance, if X is used to denote the outcome of a coin toss ("the experiment"), then the probability distribution of X would take the value 0.5 (1 in 2 or 1/2) for $X = \text{heads}$, and 0.5 for $X = \text{tails}$.

1. A **discrete probability distribution** counts occurrences that have countable or finite outcomes.

2. **Continuous probability distribution:** A probability distribution in which the random variable X can take on any value (is continuous).

3. The **central limit theorem** states that if you take sufficiently large samples from a population, the samples' means will be normally distributed, even if the population is not normally distributed.

Measure of central tendency

A measure of central tendency (also referred to as measures of center or central location) is a summary measure that attempts to describe a whole set of data with a single value that represents the middle or center of its distribution.

Why Is Central Tendency Important?

It lets us know what is normal or 'average' for a set of data. It also condenses the data set down to one representative value, which is useful when you are working with large amounts of data.

There are three main measures of central tendency which can be calculated using the methods in panda's python library.

- **Mean** - It is the Average value of the data which is a division of sum of the values with the number of values.
- **Median** - It is the middle value in distribution when the values are arranged in ascending or descending order.
- **Mode** - It is the most commonly occurring value in a distribution.

Calculating Mean and Median

The pandas functions can be directly used to calculate these values.

```
import pandas as pd
d = {'Name':pd.Series(['Tom','James','Ricky','Vin','Steve','Smith','Jack',
    'Lee','Chanchal','Gasper','Naviya','Andres']),
    'Age':pd.Series([25,26,25,23,30,29,23,34,40,30,51,46]),
    'Rating':pd.Series([4.23,3.24,3.98,2.56,3.20,4.6,3.8,3.78,2.98,4.80,4.10,3.65])}
```

```
df = pd.DataFrame(d)
print "Mean Values in the Distribution"
print df.mean()
print "*****"
print "Median Values in the Distribution"
print df.median()
```

Calculating MODE

Mode may or may not be available in a distribution depending on whether the data is continuous or whether there are values which has maximum frequency. We take a simple distribution below to find out the mode. Here we have a value which has maximum frequency in the distribution.

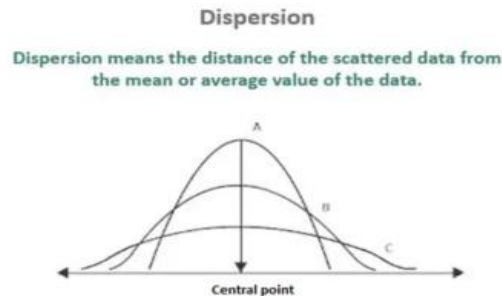
```
import pandas as pd

#Create a Dictionary of series
d = {'Name':pd.Series(['Tom','James','Ricky','Vin','Steve','Smith','Jack',
    'Lee','Chanchal','Gasper','Naviya','Andres']),
    'Age':pd.Series([25,26,25,23,30,25,23,34,40,30,25,46])}
#Create a DataFrame
df = pd.DataFrame(d)

print df.mode()
```

Dispersion for the parameters data distribution

- Dispersion parameters are a measure of how much a sample fluctuates around a mean value.
- Measurement of Central Tendency give you the information about the center of your data, dispersion measures give you the information how much your data is spread around this center.



How do you find the dispersion of a distribution?

Standard deviation (SD) is the most commonly used measure of dispersion. It is a measure of spread of data about the mean. SD is the square root of sum of squared deviation from the mean divided by the number of observations. This formula is a definitional one and for calculations, an easier formula is used.

Program to calculate SD

```
import numpy
speed = [86,87,88,86,87,85,86]
x = numpy.std(speed)
print(x)
```

EDA (Exploratory data Analysis)

- Exploratory Data Analysis (EDA) is an approach to analyze the data using visual techniques.
- It is used to discover trends, patterns, or to check assumptions with the help of statistical summary and graphical representations.
 1. Helps you prepare our dataset for analysis.
 2. Allows a machine learning model to predict our dataset better.
 3. Gives you more accurate results.
 4. It also helps us to choose a better machine learning model

Goals of EDA:

- The primary goal of EDA is to maximize the analyst's insight into a data set and into the underlying structure of a data set.
- They can extract different pieces of information, including averages, mean, minima and maxima, and other relevant points.

- More importantly, EDA can help analysts identify major errors, any anomalies, or missing values in their dataset.
- EDA helps Data Analysts and Data Scientists decide the best method for data-source manipulation to yield the most desired answers, which in turn helps locate anomalies and patterns, as well as test assumptions and hypotheses.

Univariate Analysis

What is Univariate analysis?

1. Univariate analysis is the analysis of single variable in our dataset.
2. A variable can be a column in our dataset. (or even category under which our data falls into.
 - a. Various graphs like:
 - b. Histogram
 - c. Bar chart
 - d. Density chart
 - e. Distribution plots and so on.

Program to perform univariate analysis

Import pandas as pd

Import numpy as np

Import matplotlib.pyplot as plt

Import seaborn as sns

```
Dummy_age_list= [12,22,36,42,15,89,42,65,29,6,
                  35,81,90,51,53,53,22,31,75,5]
```

```
Dummy_sex_list=['male','female','female','other',male','other','other','male',
female','female','other','male','female','female,male','female','male','female',
'male', 'other']
```

```
df_age_sex=pd.DataFrame({ 'AGE':Dummy_age_list,'Gender':Dummy_sex_list})
```

```
df_age_sex
```

Out[13]:

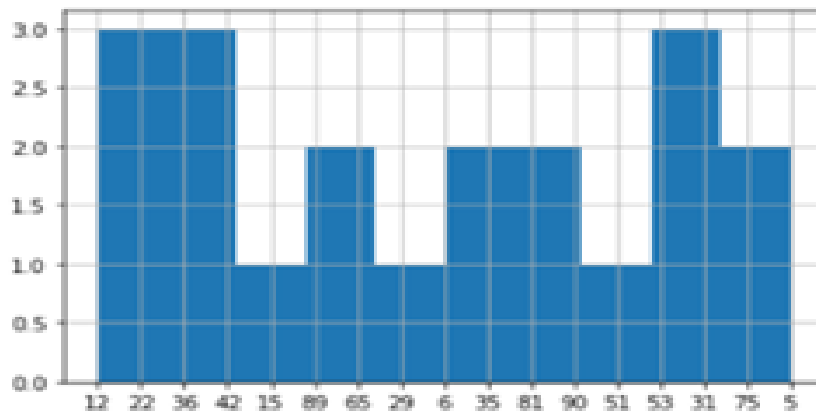
	AGE	Gender
0	12	male
1	22	female
2	36	female
3	42	other
4	15	male
5	89	other
6	42	other

Univariate distribution plot: univariate distribution plot is a **probability distribution of only one random variable**.

Univariate distribution plot can be seen by plotting histogram and bar plots. Histogram is the Frequency of values that occurs in certain range

```
In [20]: df_age_sex['AGE'].hist()
```

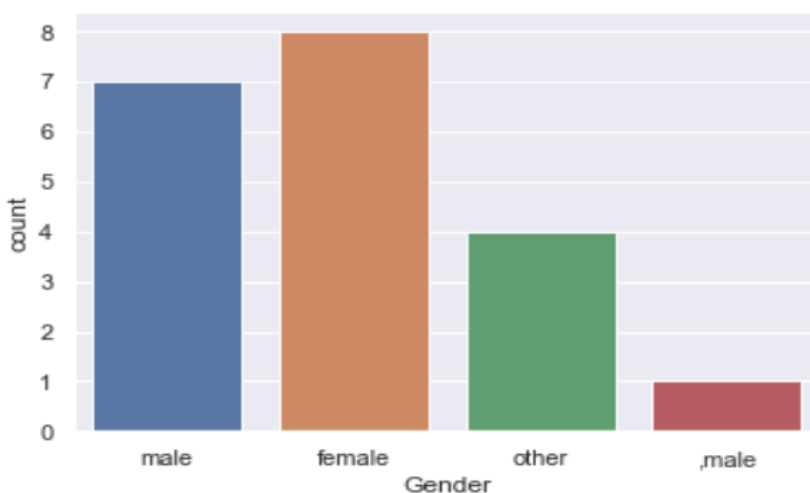
```
Out[20]: <AxesSubplot:>
```



```
In [ ]:
```

Bar charts – are used generally for categorical variables

```
] 1 sns.set(style='darkgrid')
   2 ax=sns.countplot(x='Gender',data=df_age_sex)
```



```
Maths_marks=[20,18,15,25,17,16]
```

```
Science_marks=[14,16,20,15,15,12]
```

```
Social_marks=[20,18,10,15,25,10]
```

```
Marks_df=pd.DataFrame({'maths':maths_marks  
, 'science':science_marks, 'social':social_marks})
```

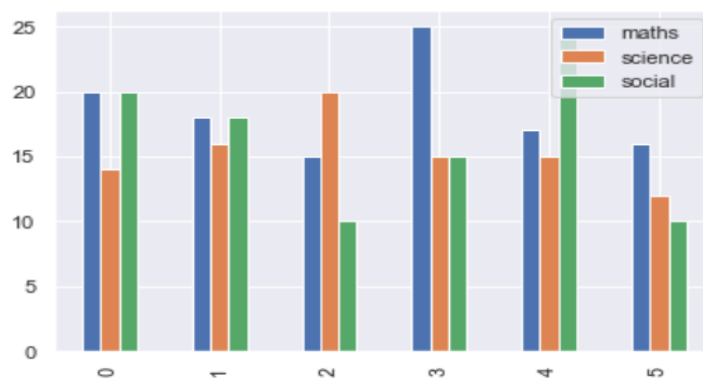
```
Print(marks_df)
```

Out[30]:

	maths	science	social
0	20	14	20
1	18	16	18
2	15	20	10
3	25	15	15
4	17	15	25
5	16	12	10

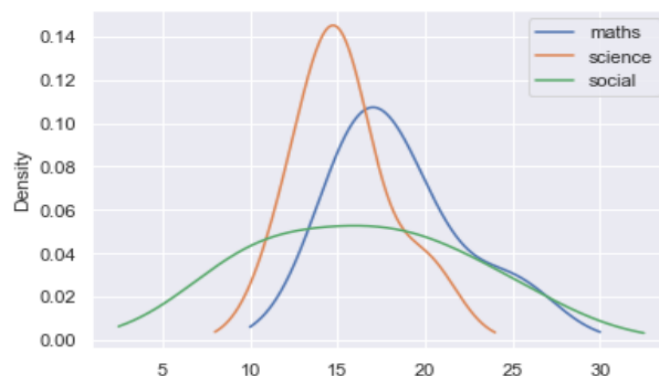
```
In [44]: marks_df.plot.bar()
```

Out[44]: <AxesSubplot:>



```
In [45]: marks_df.plot.density()
```

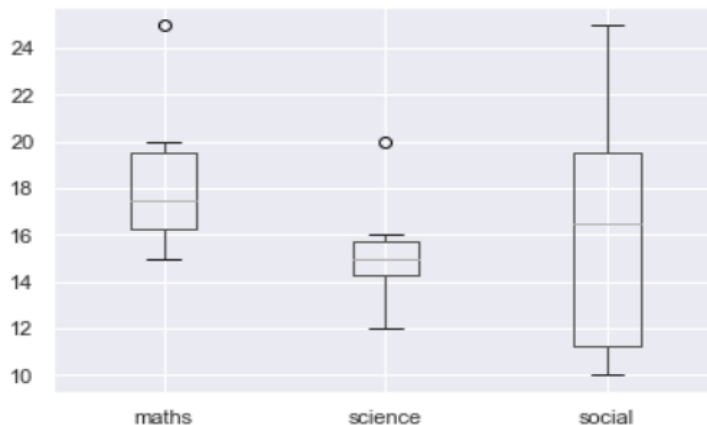
Out[45]: <AxesSubplot:ylabel='Density'>



Box and whisker plots are used to detect outlier

```
In [52]: marks_df.boxplot()
```

```
Out[52]: <AxesSubplot:>
```



Univariate analysis tests

- **Hypothesis** are statement about the given problem.
- Hypothesis testing is basically an assumption that we make about a population parameter.
- **Hypothesis testing** is done to confirm our observation about the population using sample data, within the desired error level.
- Hypothesis testing is done **to confirm our observation about the population using sample data, within the desired error level**. Through hypothesis testing, we can determine whether we have enough statistical evidence to conclude if the hypothesis about the population is true or not.

Univariate statistical tests are those tests that involve one dependent variable.

- A **test statistic** is a statistic (a quantity derived from the sample) used in statistical hypothesis testing.
- The formula for the test statistic depends on the statistical test being used. Generally, the test statistic is calculated as **the pattern in your data (i.e. the correlation between variables or difference between groups) divided by the variance in the data (i.e. the standard deviation)**

Errors in hypothesis Testing

- A type I error (false-positive) occurs if an investigator rejects a null hypothesis that is actually true in the population
- type II error (false-negative) occurs if the investigator fails to reject a null hypothesis that is actually false in the population.

P-value

- The concept of p-value comes from statistics and widely used in machine learning and data science.
- P-value is also **used as an alternative to determine the point of rejection in order to provide the smallest significance level at which the null hypothesis is least or rejected.**
- **The agreement between your calculated test statistic and the predicted values is described by the p-value.** The smaller the p-value, the less likely your test statistic is to have occurred under the null hypothesis of the statistical test.

Bivariate Analysis

- Bivariate analysis is analysing two variables or columns of a dataset to see if there is any relation between them.
- It provides broader perspective compared to univariate analysis.
- It can be done by correlation coefficients method.

Correlation Coefficient

- It is a statistical measure to understand how two variables are correlated to one another.
- It generally varies between the range of +1 to -1.
- +1 indicates that the variables are positively correlated. Meaning two variables are directly proportional.
- -1 indicates that the variables are negatively correlated. Meaning two variables are indirectly proportional.
- A value close to 0 indicates absence of any correlation.

Covariance

It is basically a measure of how two variables change together in a linear manner.

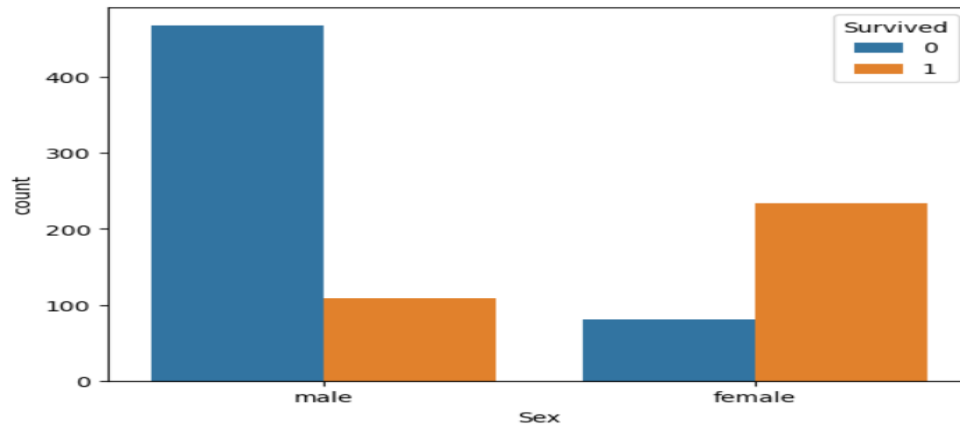
Covariance indicates the extent to which two random variables increase or decrease in tandem with each other.

Correlation tells us both the strength and the direction of this relationship. Correlation is best used for multiple variables that express a linear relationship with one another.

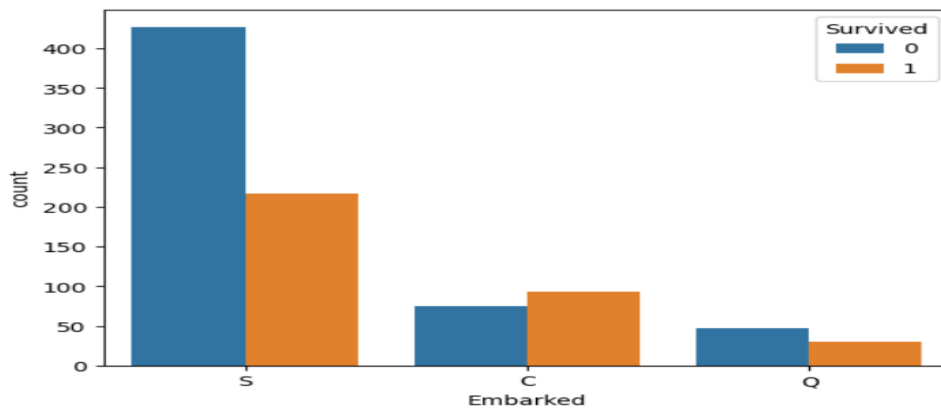
An Example of Bivariate Analysis on Titanic dataset

1. **Survived:** Outcome of survival (0 = No; 1 = Yes)
2. **Pclass:** Socio-economic class (1 = Upper class; 2 = Middle class; 3 = Lower class)
3. **Name:** Name of passenger
4. **Sex:** Sex of the passenger
5. **Age:** Age of the passenger (Some entries contain NaN)
6. **SibSp:** Number of siblings and spouses of the passenger aboard
7. **Parch:** Number of parents and children of the passenger aboard
8. **Ticket:** Ticket number of the passenger
9. ***Fare:** *Fare paid by the passenger
10. **Cabin** Cabin number of the passenger (Some entries contain NaN)
11. **Embarked:** Port of embarkation of the passenger (C = Cherbourg; Q = Queenstown; S = Southampton)

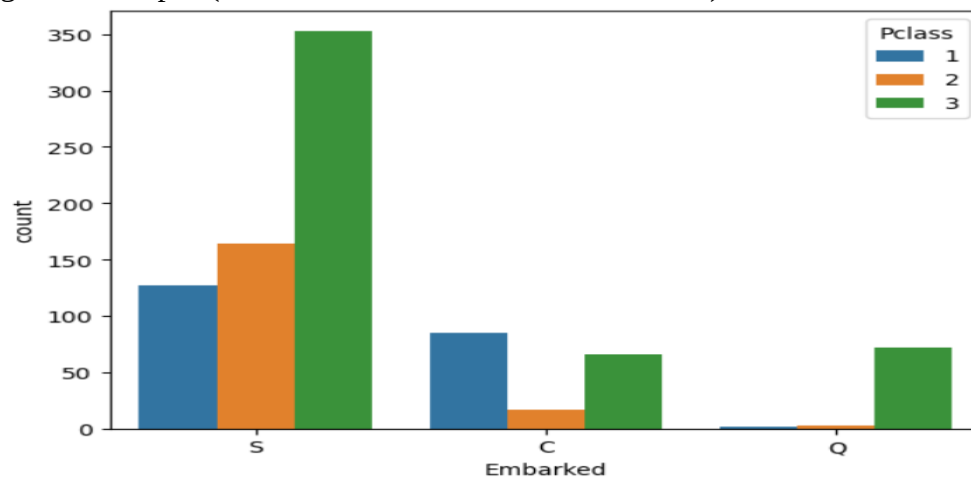
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
%matplotlib inline
data=pd.read_csv('C:/Users/Hp/Onedrive/Desktop/Datasets/train.csv')
g=sns.countplot(x='Sex', hue='Survived', data=data)
```

Output

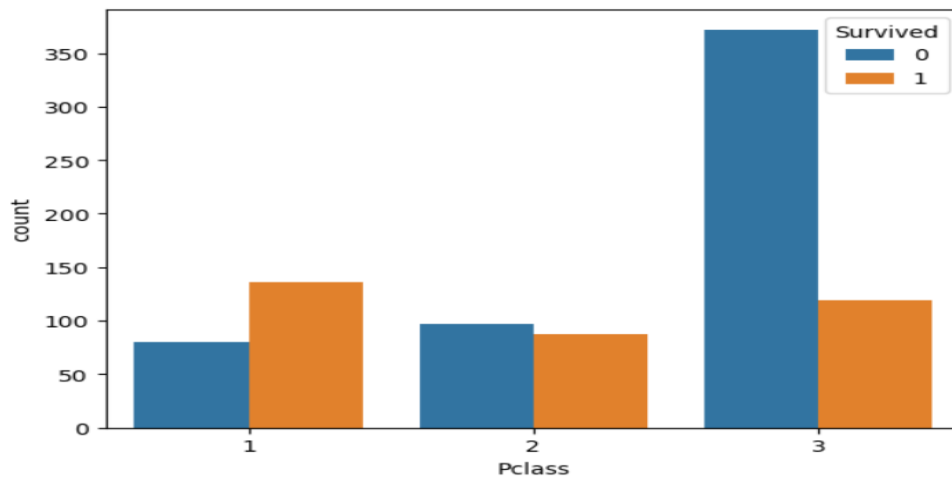
```
g=sns.countplot(x='Embarked', hue='Survived', data=data)
```



```
g=sns.countplot(x='Embarked', hue='Pclass', data=data)
```



```
g=sns.countplot(x='Pclass', hue='Survived', data=data)
```



Add family size column

```
def add_family(df):
```

```
    df['FamilySize']=df['SibSp'] + df['Parch'] +1
```

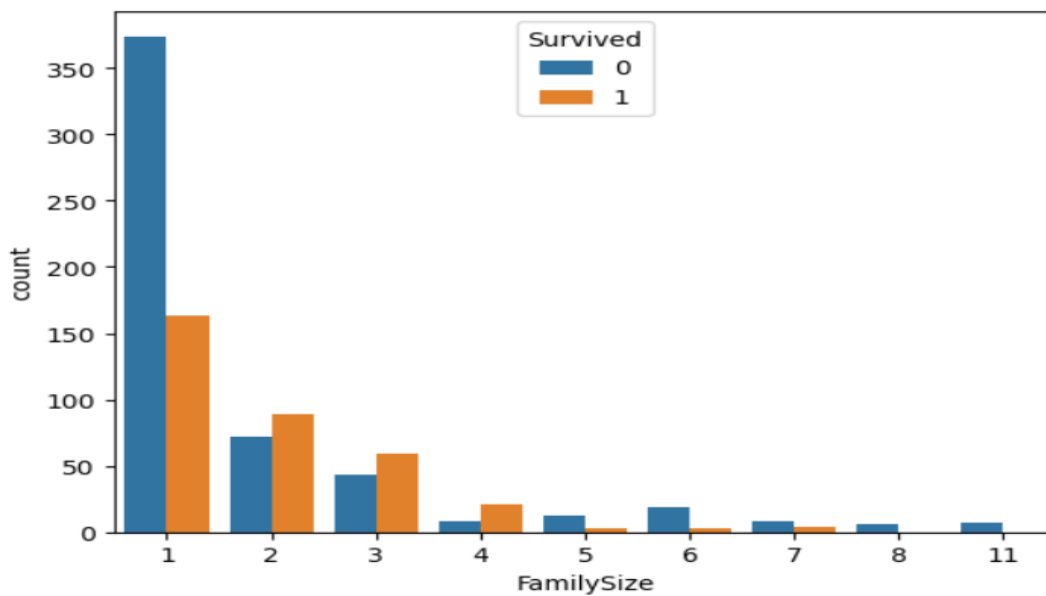
```
    return df
```

```
data=add_family(data)
```

```
data.head(10)
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked	FamilySize
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S	2
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C	2
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S	1
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S	2
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	S	1
5	6	0	3	Moran, Mr. James	male	NaN	0	0	330877	8.4583	NaN	Q	1
6	7	0	1	McCarthy, Mr. Timothy J	male	54.0	0	0	17463	51.8625	E46	S	1
7	8	0	3	Palsson, Master. Gosta Leonard	male	2.0	3	1	349909	21.0750	NaN	S	5
8	9	1	3	Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)	female	27.0	0	2	347742	11.1333	NaN	S	3
9	10	1	2	Nasser, Mrs. Nicholas (Adele Achem)	female	14.0	1	0	237736	30.0708	NaN	C	2

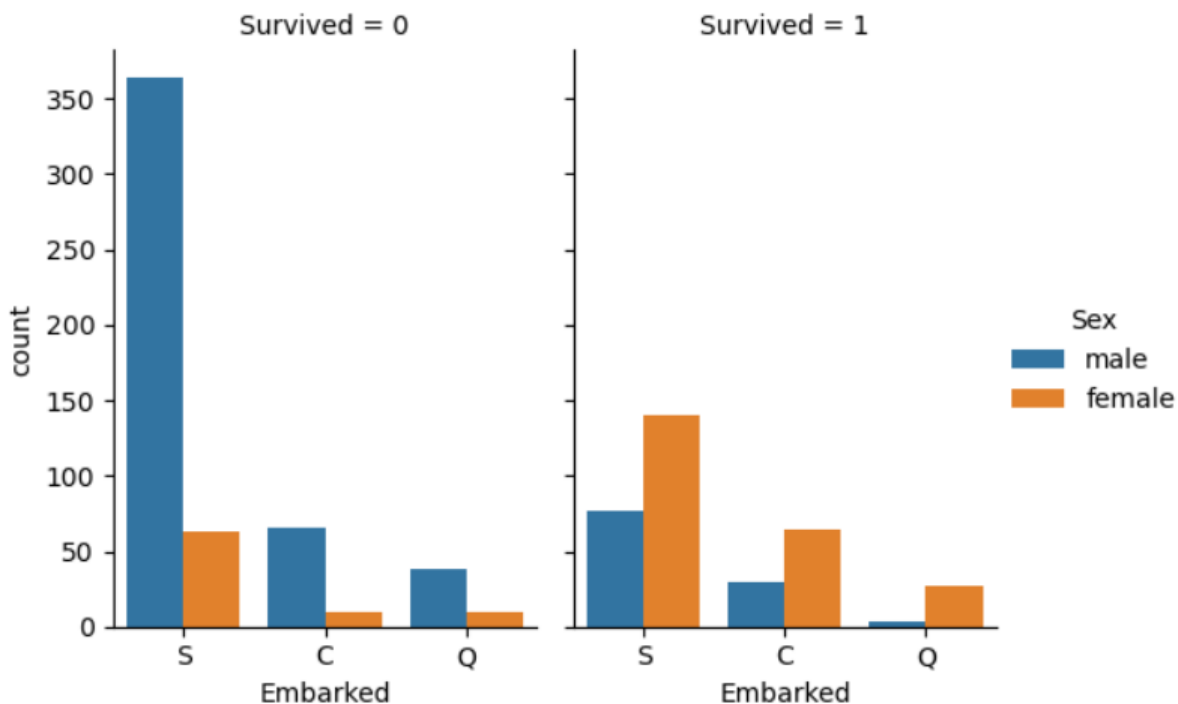
```
g=sns.countplot(x='FamilySize', hue='Survived', data=data)
```



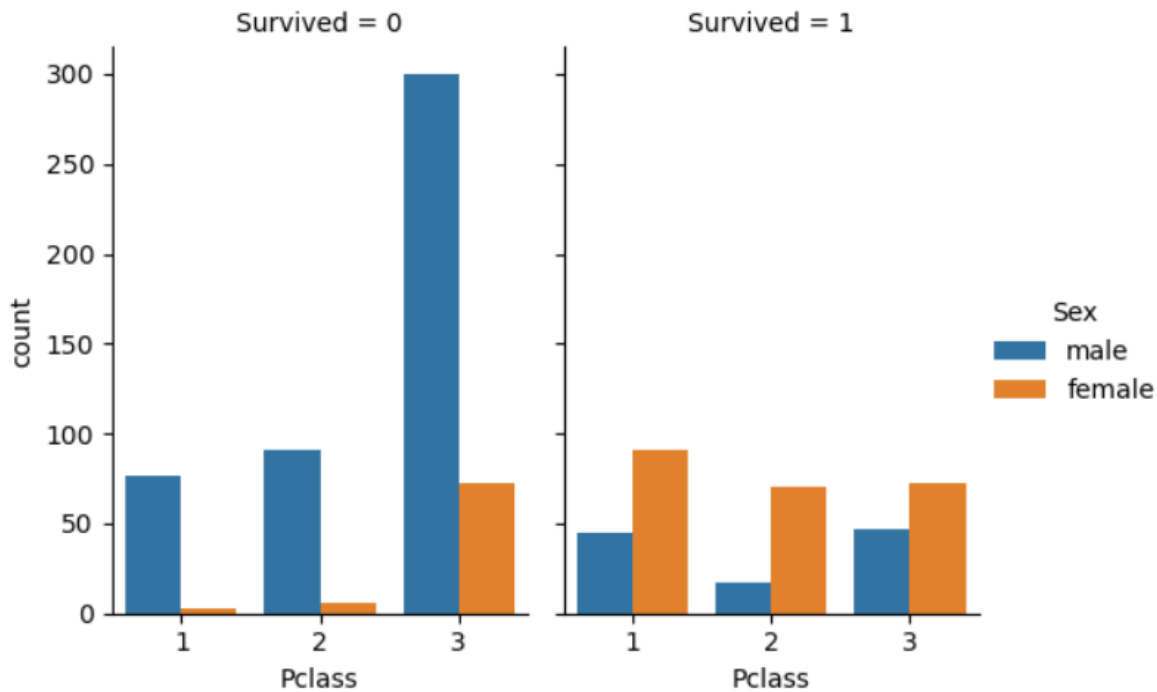
Multivariate analysis

- It can examine and analyse more than two variables at the same time.
- It provides a broader perspective as compared to bivariate or univariate analysis.
- Sometimes the underlying trends can be multidimensional in nature.
- It helps us to understand data which has complex trends based on combination of attributes.

```
g=sns.catplot(x="Embarked", hue="Sex", col="Survived", data=data, kind="count", height=4, aspect=.7)
```



```
g=sns.catplot(x="Pclass", hue="Sex", col="Survived",data=data, kind="count", height=4, aspect=.7)
```



Add FamilySize column

```
def add_family(df):
```

```
    df['FamilySize']=df['SibSp'] + df['Parch'] +1
```

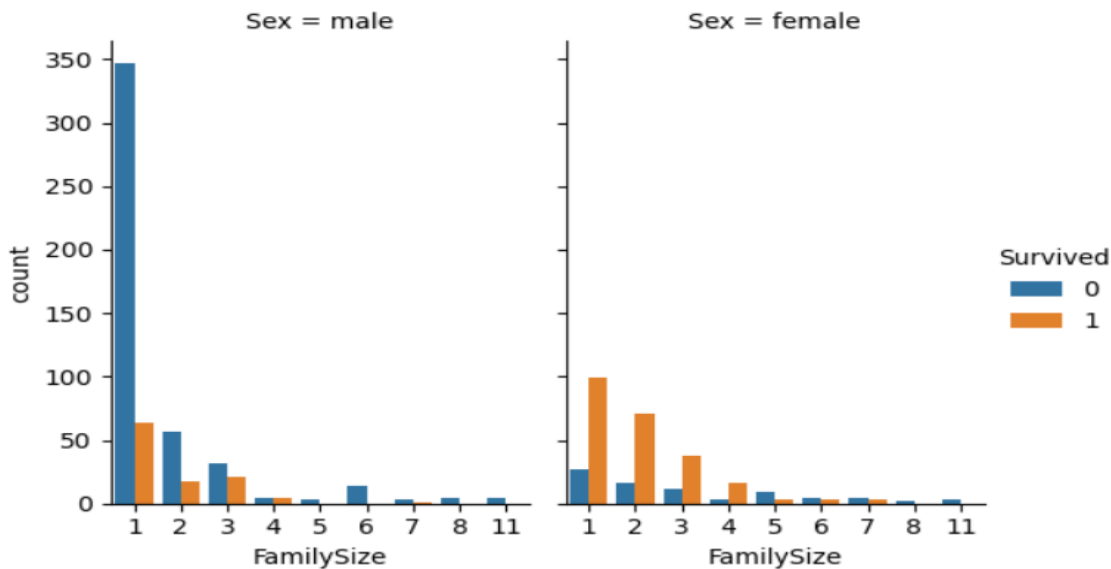
```
    return df
```

```
data=add_family(data)
```

```
data.head(10)
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked	FamilySize
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S	2
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...)	female	38.0	1	0	PC 17599	71.2833	C85	C	2
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S	1
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S	2
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	S	1
5	6	0	3	Moran, Mr. James	male	NaN	0	0	330877	8.4583	NaN	Q	1
6	7	0	1	McCarthy, Mr. Timothy J	male	54.0	0	0	17463	51.8625	E46	S	1
7	8	0	3	Palsson, Master. Gosta Leonard	male	2.0	3	1	349909	21.0750	NaN	S	5
8	9	1	3	Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)	female	27.0	0	2	347742	11.1333	NaN	S	3
9	10	1	2	Nasser, Mrs. Nicholas (Adele Achem)	female	14.0	1	0	237736	30.0708	NaN	C	2

```
g=sns.catplot(x="FamilySize", hue="Survived", col="Sex",data=data, kind="count", height=4, aspect=.7);
```



Visualization fundamentally serves 4 purposes in data science, in understanding:

- the **distribution** of features
- **relationships** between two or more variables
- **comparisons** between variables
- the **composition** of data

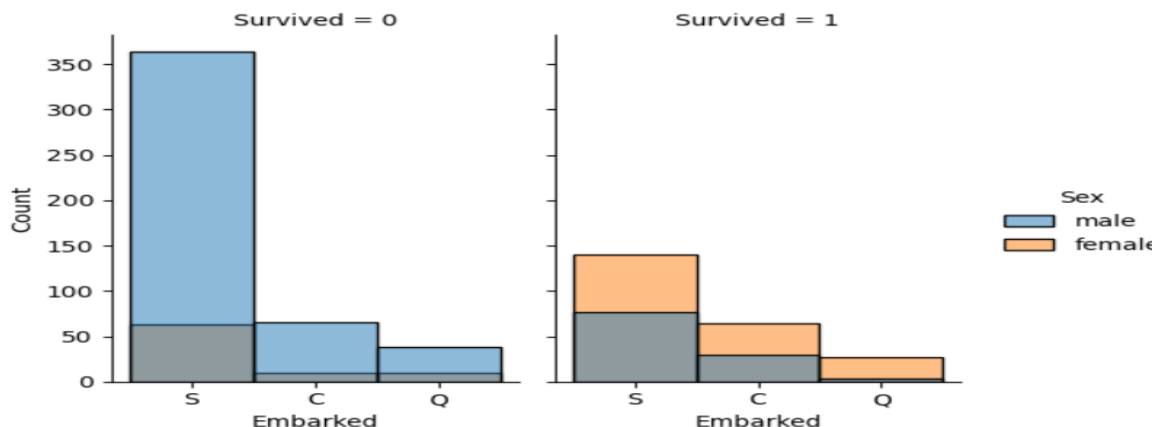
Multivariate Distribution Plot

- It allows us to understand the distribution of data. By distribution means few things: frequency distribution, probability distribution or just spread of the data with respect to central values (mean, median etc).
- **Histogram Plot is good example of distribution plot**

Displot

The displot function creates distribution plots which give us an overview of the distribution of numerical variables. We can create histograms, kde plots, ecdf plots, and rugplots with the displot function.

`g=sns.displot(x="Fare", hue="Sex", col="Survived",data=data, kind="hist", height=4, aspect=.7)`



Multivariate Relationship Plot

Relational plots display how variables are associated with each other.

Kids grow taller with age — that's a relationship between two variables: height and age.

The bigger the house, the higher the price — that's another kind of relationship between the variables: floor size and price.

Relplot

The relplot function is used to create relational plots which are line plot and scatter plot. These plots provide an overview of the relation between variables.

Multivariate Comparison Plot

The comparative analysis is key to many business decisions. Which product is selling better — product A or product B? Is there a significant difference between the two drugs in their effectiveness? These kinds of questions are asked all the time all across the decision-making landscape.

Bar plots, pair plots and violin plots are good examples of multivariate comparison plot.

Comparisonal plots

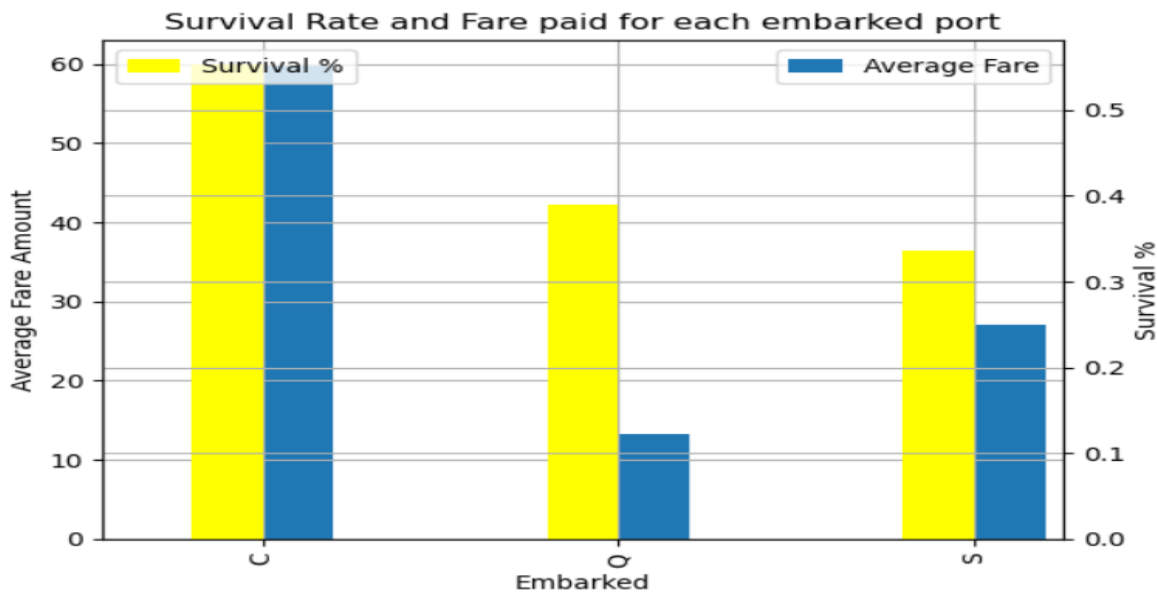
Pair plot and violin plots are used to do comparison of variables

Analysis of Average Fare Amount, Survival Rate and Embarked variables of Titanic Dataset in same graph.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

data=pd.read_csv('C:/Users/Hp/Onedrive/Desktop/Datasets/train.csv')
fig1=plt.figure()
first_axis=fig1.add_subplot()
second_axis=first_axis.twinx()
survived_df=data.groupby(['Embarked']).mean()[["Survived"]]
fare_df=data.groupby(['Embarked']).mean()[["Fare"]]
fare_df.plot(kind='bar', grid=True,ax=first_axis,width=0.2,position=0)
survived_df.plot(kind='bar',color='yellow',ax=second_axis, grid=True,width=0.2,position=1)
first_axis.set_ylabel('Average Fare Amount')
second_axis.set_ylabel('Survival %')
first_axis.legend(["Average Fare"],loc="upper right")
```

```
second_axis.legend(["Survival %"], loc="upper left")
plt.title('Survival Rate and Fare paid for each embarked port')
```



Linear Algebra

Why Linear Algebra for Data Science?

Data Science deals with extracting meaningful insights from the data. We need to represent data and perform numerous operations on them to extract insights. **Linear Algebra** helps in representation and operations on data in Data Science and Machine Learning.

Let's understand the application of Linear algebra in Data Science with the following examples.

- **Deep Learning** - Representing input to the model and model parameters as vector and matrices and making calculations using Linear Algebraic operations.
- **Image data** - Representing images as matrices and doing various geometrical transformations on them using Linear Algebraic operations.
- **Recommender system** - using linear algebraic concepts to measure similarity.
- **Multiple linear regression** - Solving multiple linear equations using linear algebra
- **Feature extraction (PCA)** - using the linear algebraic concept of eigenvalue and eigenvectors
- **One hot encoding** - A matrix representation of the encoding

What is Linear Algebra?

Linear Algebra is a branch of mathematics. It provides basic structures to represent data and various numerical methods and tools to solve problems.

The basic building blocks of Linear Algebra are scalar, vector, and matrix.

A **Scalar** is a quantity, described by a numeric value.

A **Vector** is an ordered collection of scalars.

A **Matrix** is a collection of vectors.

Scalar

1

Vector

$$\begin{bmatrix} 1 \\ 2 \end{bmatrix}$$

Matrix

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$$

An example to understand these building blocks.

Consider a student who has taken 3 exams for a given subject. The scores are 80%, 80%, and 85% respectively. Based on the difficulty level, the professor has allocated weights of 30%, 30%, and 40% to the exams respectively.

Problem

Calculate the weighted mean of the marks obtained by the student.

The weighted mean 'W' is calculated by the formula:

$$W = \frac{\sum_{i=1}^n w_i X_i}{\sum_{i=1}^n w_i}$$

where ' w_i ' represent the weights and ' X_i ' represents the corresponding marks.

Here, individual score and weightage are considered as scalars. The collection of the scores and the corresponding weights can be represented as vectors as shown below:

$x = [80, 80, 85]$

$w = [0.3, 0.3, 0.4]$

In Linear Algebra, an operation is defined called 'dot product' which helps in multiplying two vectors. The dot product of the vectors w and X is:

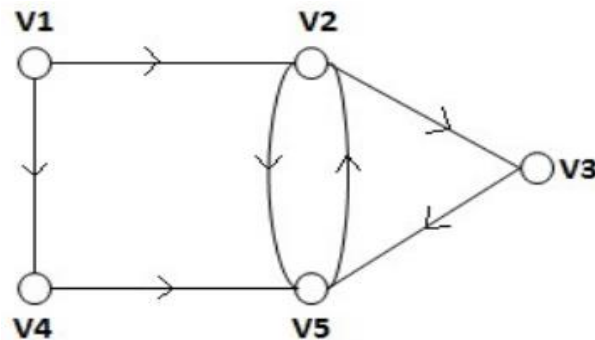
$$w \cdot X = w_1 X_1 + w_2 X_2 + w_3 X_3$$

This dot product will result in finding the weighted mean of the marks (which is a scalar).

Linear Algebraic operations are applied to the entire object (vector/matrix) instead of individual data points one at a time. This technique is called as **Vectorization** and it is more efficient while dealing with operations on large data.

Another example where Linear Algebra is used for data representation.

Consider a social networking site where five visitors are linked with each other as depicted in the graph below:



Problem

How to use these relationships to extract more information about them?

These relationships can be converted into a relationship matrix in which '1' indicates **related** and '0' indicates **not related** as shown below:

	V1	V2	V3	V4	V5
V1	0	1	0	1	0
V2	0	0	1	0	1
V3	0	0	0	0	1
V4	0	0	0	0	1
V5	0	1	0	0	0

From the above relationships, you can create a matrix for the directed graph as below:

$$\begin{bmatrix} 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 & 0 \end{bmatrix}$$

This matrix can be used as a data structure for representing graphs in computer programs for further computation.

Scalar

A **Scalar** is a quantity which is described by a numerical value. It is used to represent values such as:

- A number like 50
- Mass of sun in kilograms
- Length of the great wall of china in kilometres

Define scalars

```
number_of_units = 50
mass_of_sun = 1.989e30
length_great_wall_of_china = 21196
print(number_of_units)
print(mass_of_sun)
print(length_great_wall_of_china)
```

OUTPUT:

```
50
1.989e+30
21196
```

Vector

A vector is an ordered collection of scalars. It can be represented in two forms:

- **Column vector:** Contains n rows and 1 column
- **Row vector:** Contains 1 row and n columns

The number of scalars in a vector is called its **dimension** or **order**.

Example: A column vector 'u' of dimension 2 is illustrated below:

$$u = \begin{bmatrix} 4 \\ 3 \end{bmatrix}$$

Example: A row vector 'v' of dimension 3 is illustrated below:

$$v = \begin{bmatrix} 4 & -1 & 2 \end{bmatrix}$$

Row and column vectors can be created using the library NumPy as follows:

```
# importing required library (numpy as np)
import numpy as np
# creating a column vector with 2 rows and 1 column using np.array() function
col_vec = np.array([[4],[3]])
print("A column vector\n",col_vec)
# printing the shape of the vector
print("Shape of vector=",col_vec.shape)
```

OUTPUT:

```
A column vector
[[4]
 [3]]
Shape of vector= (2, 1)
```

Create Row Vector

```
# importing required library (numpy as np)

import numpy as np

# creating a row vector with 1 row and 3 columns using np.array() function
row_vec = np.array([[4,-1,2]])

print("A row vector\n",row_vec)

# printing the shape of the vector
print("Shape of vector=",row_vec.shape)
```

OUTPUT:

```
A row vector
[[ 4 -1  2]]
Shape of vector= (1, 3)
```

Zero vector (z) is a vector in which each element is zero.

One vector (o) is a vector in which each element is one.

$$z = \begin{bmatrix} 0 & 0 & 0 \end{bmatrix} \quad o = \begin{bmatrix} 1 & 1 & 1 \end{bmatrix}$$

Zero and one vectors can be created using the library NumPy as follows:

```
# importing required library (numpy as np)

import numpy as np

#Illustrating a Zero row vector of dimension 3 using np.array() function
zero_vector_1=np.array([0,0,0])

print(zero_vector_1)

#Illustrating a Zero row vector of dimension 3 using np.zeros() function
zero_vector_2=np.zeros(3)

print(zero_vector_2)

#Illustrating a Zero column vector of dimension 3 using np.zeros() and reshape() function
zero_vector_3=np.zeros(3).reshape(3,1)
```

```
print(zero_vector_3)
```

OUTPUT:

```
[0 0 0]
[0. 0. 0.]
[[0.]
 [0.]
 [0.]]
```

Create and print One Vector using various functions

```
# importing required library (numpy as np)
```

```
import numpy as np
```

```
#Illustrating a One vector of dimension 3 using np.array() function
```

```
one_vector_1=np.array([1,1,1])
```

```
print(one_vector_1)
```

```
#Illustrating a One vector of dimension 3 using np.ones() function
```

```
one_vector_2=np.ones(3)
```

```
print(one_vector_2)
```

```
#Illustrating a column vector of ones of dimension 3 using np.ones() and reshape() function
```

```
one_vector_3=np.ones(3).reshape(3,1)
```

```
print(one_vector_3)
```

OUTPUT:

```
[1 1 1]
[1. 1. 1.]
[[1.]
 [1.]
 [1.]]
```

Matrix

A **matrix** is defined as a rectangular array of numbers. It consists of rows and columns.

The number of rows (m) and columns (n) is defined as the **dimension or order** of the matrix and is represented as (m, n). A matrix is called a **square matrix** if the number of rows(m) is equal to the number of columns(n).

Using the library NumPy, you can create matrices of different orders as shown below:

```
# importing required libraries (numpy as np)
```

```
import numpy as np
```

```
# Creating matrices of various dimension using np.array() function
```

```
matrix_2x2 = np.array([[1, 2],
                       [3, 4]])
```

```
matrix_3x3 = np.array([[1, 2, 3],
                       [4, 5, 6],
                       [8, 9, 10]])
```

```
print("2x2 Matrix\n",matrix_2x2,"\nshape of matrix -> ",matrix_2x2.shape)
```

```
print("\n3x3 Matrix\n",matrix_3x3,"\nshape of matrix -> ",matrix_3x3.shape)
```

OUTPUT:

```
2x2 Matrix
[[1 2]
 [3 4]]
shape of matrix -> (2, 2)

3x3 Matrix
[[ 1  2  3]
 [ 4  5  6]
 [ 8  9 10]]
shape of matrix -> (3, 3)
```

Accessing elements of a matrix

Consider the following matrix A,

$$A = \begin{bmatrix} 2 & -2 \\ 3 & 1 \\ 1 & 4 \end{bmatrix}$$

```
# importing required libraries (numpy as np)
```

```
import numpy as np
```

```
# Creating a 3x2 Matrix "A"
```

```
A = np.array([[2,-2],[3,1],[1,4]])
```

Recall that, indexing starts from 0. Elements of matrix A can be accessed using row-id and column-id as shown below:

```
#selecting an element at row index 2 and column index 1
```

```
print("Element of matrix A at index position [2,1] is", A[2,1])
```

OUTPUT:

```
Element of matrix A at index position [2,1] is 4
```

Transpose of a matrix

Transpose of a matrix is obtained by interchanging its rows and columns.

For matrix A,

$$A = \begin{bmatrix} 2 & -2 \\ 3 & 1 \\ 1 & 4 \end{bmatrix}$$

the transposed matrix (A^T) is given below,

$$A^T = \begin{bmatrix} 2 & 3 & 1 \\ -2 & 1 & 4 \end{bmatrix}$$

NumPy library has a function **transpose()** which is used to obtain the transpose of a given matrix.

An example is illustrated below:

```
# importing required libraries (numpy as np)
```

```
import numpy as np
```

```
# Creating a 3x2 Matrix "A"
```

```
A = np.array([[2,-2],[3,1],[1,4]])
```

```
print("Matrix A:\n",A)
```

```
# Finding transpose of a matrix using the function transpose()
```

```
print("Matrix A Transpose: \n",np.transpose(A))
```

OUTPUT:

```

Matrix A:
[[ 2 -2]
 [ 3  1]
 [ 1  4]]
Matrix A Transpose:
[[ 2  3  1]
 [-2  1  4]]

```

Types of Matrices

Matrices can be categorized on the basis of the value of their elements, position of elements, number of rows and columns, etc. as follows:

- Diagonal matrix
- Identity matrix
- Symmetric matrix
- Triangular matrix

Diagonal Matrix

A Square Matrix (D) is called a Diagonal Matrix, if D has zeros outside the main diagonal or principal diagonal. The Main diagonal or the principal diagonal are the elements on the diagonal that runs from the top left to bottom right.

Example: The matrix 'A' shown below illustrates a (3, 3) Diagonal matrix.

$$A = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 3 & 0 \\ 0 & 0 & 2 \end{bmatrix}$$

```

# importing required libraries (numpy as np)
import numpy as np
# Creating a diagonal matrix with diagonal elements as (1,3,2)
diagonal_matrix = np.diag((1,3,2))
print(diagonal_matrix)

```

Identity Matrix

An **identity matrix** is a square matrix of dimensions (n, n) having '1' across its main diagonal and '0' everywhere else. It is usually represented as 'I_n'

Example: The matrix shown below illustrates a (3, 3) Identity matrix.

$$I_3 = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

In NumPy, there are many methods to create an identity matrix

```

# importing required libraries (numpy as np)
import numpy as np
#Creating an Identity matrix of size 2 using np.identity() function
identity_matrix_1 = np.identity(2)
print("Identity_matrix 1\n",identity_matrix_1)
#Creating an Identity matrix of size 3 using np.eye() function

```

```
identity_matrix_2 = np.eye(3)
print("\nIdentity_matrix 2\n", identity_matrix_2)
```

OUTPUT:

```
Identity_matrix 1
[[1. 0.]
 [0. 1.]]

Identity_matrix 2
[[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]
```

Symmetric Matrix

A square matrix A of dimension (n, n) is symmetric, if $A = A^T$ i.e. the matrix A is the same as its transpose.

Example: Consider the following matrix A :

$$A = \begin{bmatrix} a_{11} & b & c \\ b & a_{22} & d \\ c & d & a_{33} \end{bmatrix}$$

The Transpose of A (A^T) is shown below:

$$A^T = \begin{bmatrix} a_{11} & b & c \\ b & a_{22} & d \\ c & d & a_{33} \end{bmatrix}$$

Since $A = A^T$, A is a **symmetric matrix**.

importing required libraries (numpy as np)

import numpy as np

#Creating matrix A

```
A = np.array([[2,3,1],
              [3,4,-1],
              [1,-1,1]])
```

```
print("A:\n", A)
```

Finding the Transpose of the matrix

```
transposed_matrix = A.transpose()
```

```
print("Transpose of A:\n", transposed_matrix)
```

#Comparing each element of both matrices (returns a matrix of boolean compared values) and saving it in a variable comparison

```
comparison = (A == transposed_matrix)
```

#Checking if all the elements in the matrix comparison is true

```
equal_arrays = comparison.all()
```

```
print(equal_arrays)
```

Triangular Matrix

A **triangular matrix** can be either a lower triangular or an upper triangular matrix.

A **lower triangular matrix** is a square matrix in which all the elements above the main diagonal are zero.

Example: L is a lower triangular matrix of dimension $(3, 3)$

$$L = \begin{bmatrix} a_{11} & 0 & 0 \\ a_{12} & a_{22} & 0 \\ a_{13} & a_{23} & a_{33} \end{bmatrix}$$

An **upper triangular matrix** is a square matrix in which all the elements below the main diagonal are zero.

Example: U is an upper triangular matrix of dimension (3, 3)

$$U = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ 0 & a_{22} & a_{23} \\ 0 & 0 & a_{33} \end{bmatrix}$$

In NumPy, the functions **np.tril()** and **np.triu()** are used to create a lower triangular matrix and an upper triangular matrix, respectively.

```
# importing required libraries (numpy as np)
import numpy as np
#Creating a lower triangular matrix
lower_triangular_matrix_1 = np.tril([[1,2,3],
                                     [4,5,6],
                                     [7,8,9]])
print("Lower triangular matrix\n",lower_triangular_matrix_1)
```

OUTPUT:

```
Lower triangular matrix
[[1 0 0]
 [4 5 0]
 [7 8 9]]
```

```
# importing required libraries (numpy as np)
import numpy as np
#Creating a Upper triangular matrix
upper_triangular_matrix_1 = np.triu([[1,2,3],
                                     [4,5,6],
                                     [7,8,9]])
print("Upper triangular matrix\n",upper_triangular_matrix_1)
```

OUTPUT:

```
Upper triangular matrix
[[1 2 3]
 [0 5 6]
 [0 0 9]]
```

Operations over vectors and matrices

Vectors of the same dimension can be added by adding their corresponding elements.

The function **add()** of the NumPy library, is used to add vectors. Vector addition is illustrated below:

```
# importing required libraries (numpy as np)
import numpy as np
#Create 2 vectors
vector_1 = np.array([[2, -1, 1]])
vector_2 = np.array([[1, 2, -1]])
print("1st Vector : ", vector_1)
print("2nd Vector : ", vector_2)
# Addition of the vectors using the function np.add()
out = np.add(vector_1, vector_2)
print("Added Vector : ", out)
```

OUTPUT:

```
1st Vector :  [[ 2 -1  1]]
2nd Vector :  [[ 1  2 -1]]
Added Vector : [[3  1  0]]
```

Matrix Addition

Matrices of the same dimension can be added by adding their corresponding elements.

```
# importing required libraries (numpy as np)
import numpy as np
# Creation of 2 matrices
matrix_1 = np.array([[10,20,30],
                    [-30,-40,-50]])
matrix_2 = np.array([[100,-200,300],
                    [30,50,70]])
print("1st Matrix : \n", matrix_1)
print("2nd Matrix : \n", matrix_2)
# Addition of the matrices using np.add() function
out = np.add(matrix_1, matrix_2)
print("Added Matrix : \n", out)
```

OUTPUT:

```
1st Matrix :
[[ 10  20  30]
 [-30 -40 -50]]
2nd Matrix :
[[ 100 -200  300]
 [  30   50   70]]
Added Matrix :
[[ 110 -180  330]
 [   0   10   20]]
```

Multiplication of Vector by a Scalar

In the multiplication of a vector by a scalar, each element of the vector is multiplied by the given scalar quantity.

The scalar product of s and v ($s \cdot v$) is as given below:

$$s \cdot v = s \cdot \begin{bmatrix} a_1 \\ a_2 \\ a_3 \end{bmatrix} = \begin{bmatrix} s \cdot a_1 \\ s \cdot a_2 \\ s \cdot a_3 \end{bmatrix}$$

```
# importing required libraries (numpy as np)
import numpy as np
#Defining a Vector 'v' and scalar 's'
v = np.array([[2],[-1],[3]])
s = 2.5
#Scalar Vector Multiplication
vector_mul = v*s
print("Vector: \n",v,"\nScalar:",s,"\nScalar Vector multiplication:\n",vector_mul)
```

OUTPUT:

```
Vector:
[[ 2]
 [-1]
 [ 3]]
Scalar: 2.5
Multiplication Of Vector by a Scalar:
[[ 5.]
 [-2.5]
 [ 7.5]]
```

Multiplication of Matrix by a Scalar

In the multiplication of a matrix by a scalar, each element of the matrix is multiplied by the given scalar quantity.

```
# importing required libraries (numpy as np)
import numpy as np
#Defining a matrix 'A' and scalar 's'
A = np.array([[1,2,1],
              [-1,1,0],
              [2,-1,1]])
print("Matrix:\n",A)
s = 2.5
print("Scalar :",s)
#Multiplication of Matrix by a Scalar
matrix_mul = s * A
```

```
print("Multiplication of Matrix by a Scalar:\n",matrix_mul)
```

OUTPUT:

```
Matrix:
[[ 1  2  1]
 [-1  1  0]
 [ 2 -1  1]]
Scalar : 2.5
Multiplication of Matrix by a Scalar:
[[ 2.5  5.  2.5]
 [-2.5  2.5  0. ]
 [ 5.  -2.5  2.5]]
```

Orthogonal vectors

If the inner product of two non-zero vectors v_1 and v_2 is zero, that is

$$\langle v_1, v_2 \rangle = 0$$

then, the vectors v_1 and v_2 are called **orthogonal** vectors.

```
# importing required libraries (numpy as np)
```

```
import numpy as np
```

```
#Creating vectors
```

```
Vector_1 = np.array([[3],[-1],[2]])
```

```
Vector_2 = np.array([[2],[4],[-1]])
```

```
print("Vector 1\n",Vector_1)
```

```
print("Vector 2\n",Vector_2)
```

```
#Finding the transpose of Vector_1
```

```
trans = np.transpose(Vector_1)
```

```
#Finding the dot product
```

```
result = np.dot(trans,Vector_2)
```

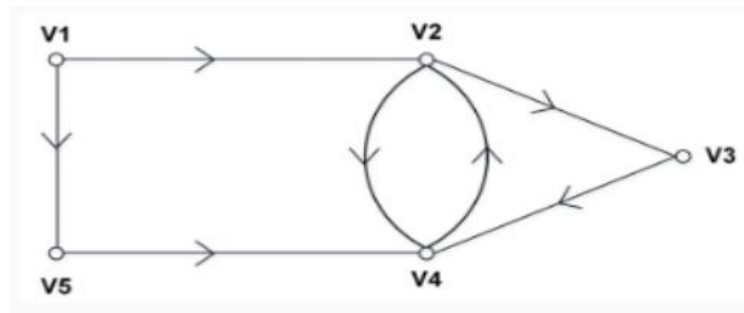
```
print("Dot Product\n",result)
```

OUTPUT

```
Vector 1
[[ 3]
 [-1]
 [ 2]]
Vector 2
[[ 2]
 [ 4]
 [-1]]
Dot Product
[[0]]
```

Application of Matrix Multiplication

Consider a social networking site where five people are linked with each other as depicted in the graph below.



From the graph, you can see that there is a direct link from V1 to V2 (V1 → V2). Therefore, V1 and V2 are friends or V2 is at distance 1 from V1. The nodes V1 and V3 are not connected through a direct link. They are connected through V2 i.e. V1 → V2 → V3. So, V3 is a friend of a friend of V1 and distance of V3 from V1 is 2.

This given network can be represented using an adjacency matrix as shown below:

$$A = \begin{bmatrix} 0 & 1 & 0 & 0 & 1 \\ 0 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \end{bmatrix}$$

Problem

From the graph, find number of paths of length 2 between every pair of vertices.

Solution

Matrix multiplication can solve this problem by multiplying the matrix by itself.

Multiply the matrix A with itself:

$$A * A = \begin{bmatrix} 0 & 1 & 0 & 0 & 1 \\ 0 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \end{bmatrix} * \begin{bmatrix} 0 & 1 & 0 & 0 & 1 \\ 0 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \end{bmatrix} = \begin{matrix} & \begin{matrix} v1 & v2 & v3 & v4 & v5 \end{matrix} \\ \begin{matrix} v1 \\ v2 \\ v3 \\ v4 \\ v5 \end{matrix} & \begin{pmatrix} 0 & 0 & 1 & 2 & 0 \\ 0 & 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 \\ 0 & 1 & 0 & 0 & 0 \end{pmatrix} \end{matrix}$$

In the resultant matrix, the value in the first row and the third column is 1. This represents that there is one path of length 2 between the vertices V1 and V3 and the path is:

V1 → V2 → V3

In the resultant matrix, the value in the first row and the fourth column is 2. This represents that there are two different paths of length 2 between the vertices V1 and V4, i.e.

1. V1 -> V2 -> V4
2. V1 -> V5 -> V4

The other non-zero values of the resultant matrix can be interpreted in similar manner.

Gradients

In the following, we consider the general case where the function f depends on one or more variables $x \in \mathbb{R}^n$, e.g., $f(x) = f(x_1, x_2)$. **The generalization of the derivative to functions of several variables is the gradient.** We find the gradient of the function f with respect to x by varying one variable at a time and keeping the others constant. The gradient is then the collection of these partial derivatives.

Tensors

Tensors are a type of data structure used in linear algebra, and like vectors and matrices, you can calculate arithmetic operations with tensors.

The extension of matrix-like structure to higher-dimensional arrays are called tensors.

```
1# create tensor from numpy import array
3T = np.array([
4 [[1,2,3], [4,5,6], [7,8,9]],
5 [[11,12,13], [14,15,16], [17,18,19]],
6 [[21,22,23], [24,25,26], [27,28,29]],
7 ])
8print(T.shape)
9print(T)
```

OUTPUT

(3, 3, 3)

```
[[[ 1  2  3]
   [ 4  5  6]
   [ 7  8  9]]
```

```
[[11 12 13]
 [14 15 16]
 [17 18 19]]
```

```
[[21 22 23]
 [24 25 26]
 [27 28 29]]]
```

Eigen values and eigen vectors

A **linear map** is a mapping between two vector spaces that preserves the operations of vector addition and scalar multiplication.

Eigen values and vectors are a new way to characterize a matrix and its associated linear mapping.

Every linear mapping has a unique transformation matrix given an ordered basis. We can interpret linear mappings and their associated transformation matrices by performing an “eigen” analysis.

Eigen is a German word meaning “characteristic”, “self”, or “own”.

The eigenvalue equation.

Let $A \in \mathbb{R}^{n \times n}$ be a square matrix. Then $\lambda \in \mathbb{R}$ is an eigenvalue of A and $x \in \mathbb{R}^n \setminus \{0\}$ is the corresponding eigenvector of A if $Ax = \lambda x$.

Compute Eigen Values

Example

$$A = \begin{bmatrix} 3 & 1 \\ 0 & 3 \end{bmatrix}$$

Triangular Matrix

$$\det(A - \lambda I)$$

$$= \begin{vmatrix} 3-\lambda & 1 \\ 0 & 3-\lambda \end{vmatrix}$$

$$= (3-\lambda)^2 - 1 \cdot 0$$

$$= (3-\lambda)^2$$

Only has 1 eigenvalue!

$$\therefore \lambda_1 = \lambda_2 = 3$$

Example 2

Let us find the eigenvalues and eigenvectors of the 2×2 matrix

$$A = \begin{bmatrix} 4 & 2 \\ 1 & 3 \end{bmatrix}.$$

$$\det(A - \lambda I)$$

$$\det\left(\begin{bmatrix} 4 & 2 \\ 1 & 3 \end{bmatrix} - \begin{bmatrix} \lambda & 0 \\ 0 & \lambda \end{bmatrix}\right) = \begin{vmatrix} 4-\lambda & 2 \\ 1 & 3-\lambda \end{vmatrix}$$

$$(4-\lambda)(3-\lambda) - 2 \cdot 1.$$

We factorize the characteristic polynomial and obtain

$$p(\lambda) = (4-\lambda)(3-\lambda) - 2 \cdot 1 = 10 - 7\lambda + \lambda^2 = (2-\lambda)(5-\lambda)$$

giving the roots $\lambda_1 = 2$ and $\lambda_2 = 5$.

Program to calculate eigen values and eigen vectors

```
import numpy as np
a = np.array([[4,2], [1, 3]])
w, v = np.linalg.eig(a)
print("Eigen Values are:",w)
print("Eigen Vectors are:",v)
```

Output

```
Eigen Values are: [5.  2.]
Eigen Vectors are: [[ 0.89442719 -0.70710678]
 [ 0.4472136   0.70710678]]
```